

Fine-tuning, Reinforcement Learning and Parameter-Efficient Adaptation

Dimitris Tsirmpas

Athens University of Economics and Business

Archimedes, Athena Research Center | June 17, 2026

Why finetune LLMs?

- **Prompting:** Changes **input behavior**. We guide the model with instructions and examples.
- **Fine-Tuning:** Changes **model parameters**. We switch the **knowledge** of the model.

Reminder

Fine-Tuning (FT): The process of further training a pre-trained model on a smaller, domain-specific dataset to adapt its general knowledge to a specialized task.

Why not finetune LLMs?

- **High Resource Cost:**
 - **Computational Power:** Full FT requires massive and thousands of compute hours for large models (e.g., Llama-70B).
 - **Storage:** Each specialized task requires a separate, full copy of the massive foundational model.

Why not finetune LLMs?

- **Operational Overhead:**

- Training is **slow**.
- Deployment becomes complex, managing **multiple large models** for different applications.
 - A single 13B model may require 26GB of storage.
 - 10 finetuned models may thus require 260GB.
- High risk of **Catastrophic Forgetting**—the model forgets its general knowledge while learning the specific task.

Why not finetune LLMs?

BERT

- **Model Size:** ~340M Parameters
- **VRAM Requirement:** $\approx$ 8-12GB
- **Training Time:** Minutes to a few hours

Llama 70B

- **Model Size:** ~70 Billion Parameters
- **VRAM Requirement:** $\approx$ 1000GB+
- **Training Time:** Days to Weeks (even with specialized hardware)

LLMs are **too large** to run full-finetuning. But do we need to?

Parameter-Efficient Fine-Tuning (PEFT)

- Instead of finetuning the whole model, why not finetune **only a small part**?
 - Similar in concept to finetuning a classification head.
 - Introduce and train a tiny number of new, small matrices.
- **Goal:** Achieve performance benefits of Fine-Tuning while **drastically reducing the computational burden**.

PEFT: LoRA

- Low-Rank (LoRA) Matrix Decomposition is a specific PEFT algorithm that exploits the fact that weight updates in LLMs have a “low intrinsic rank”.
- Low rank matrices enable us to **scalably** represent information.

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 1 & 2 & 3 & 4 & 5 \\ 3 & 6 & 9 & 12 & 15 \\ 4 & 8 & 12 & 16 & 20 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 3 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \end{bmatrix}.$$

LoRA: Low-rank decomposition

$$\underbrace{A_{m \times n}}_{\text{large matrix}} \approx \underbrace{U_{m \times r}}_{\text{left factors}} \underbrace{V_{r \times n}}_{\text{right factors}}, \quad r \ll \min(m, n).$$

$$A_{10000 \times 5000} \approx U_{10000 \times 50} V_{50 \times 5000},$$

$$\underbrace{10000 \times 5000}_{50,000,000 \text{ parameters}} \longrightarrow \underbrace{10000 \times 50 + 50 \times 5000}_{750,000 \text{ parameters}}.$$

LoRA: Mathematical Foundation

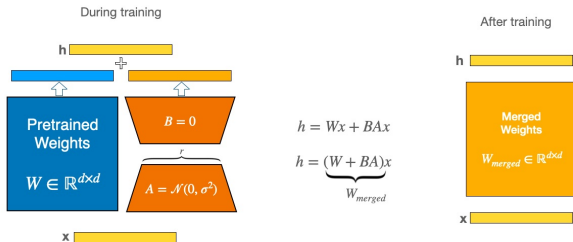


Figure: https://huggingface.co/docs/peft/main/conceptual_guides/lora

- $W \in \mathbb{R}^{d \times k}$ weights.
- x input, h output.
- We add BAx to the output.
- $B \in \mathbb{R}^{d \times r} \rightarrow$ zeros.
- $A \in \mathbb{R}^{r \times k} \rightarrow$ Gaussian.
- $r \ll \min(d, k)$, hence A, B are much smaller than W .

LoRA: Algorithm

▷ *====Initialization=====*

◁

for all $W \in all_weights$ **do**

└ Set W to read-only ▷ *Original knowledge preserved*

for all $W_u \in target_modules$ **do**

└ Initialize A randomly

└ Initialize $B = 0$ ▷ *Contributes zero to the output initially*

▷ *====Backpropagation=====*

◁

for all $X \in input_embeds$ **do**

└ $\Delta W = \frac{\alpha}{r} AB$

└ $H = WX + \Delta WX$ ▷ *Output = original + adaptation*

▷ *====End of training=====*

◁

$W' = W + \frac{\alpha}{r}(AB)$ ▷ *Merge original and adaptation*

LoRA: Practical Considerations

- How many ranks do I use?
 - $r \in [1, 16]$: fast, easy, recommended for minor adjustments.
 - $r \in [16, 64]$: recommended for most domain adaptation scenarios.
 - $r \in [64, 256]$: may hit computational bottlenecks, only use for dramatic shifts in domain and knowledge.
- How do I set α ?
 - Usually set to a multiple of r .
 - Very low $\alpha \rightarrow$ minimal contribution. Very high $\alpha \rightarrow$ destruction of model knowledge.
 - $\alpha = 2r$ usually used as a starting point.

LoRA: Practical Considerations

- What modules do I change?
- Start from the top of this list and go down as required:
 1. **Query (q_proj) and Value (v_proj):** Most frequently adapted; control **how the model attends** to input.
 2. **Key (k_proj):** Increases **information retrieval** capacity.
 3. **Output (o_proj):** Less common, but useful for modifying **integrated** attended information.
 4. **Feed-forward Networks (MLP):** Possible to adapt, but generally offer less parameter benefit compared to attention layers.

LoRA: Limitations

- The largest limitation to LoRA is **memory** and **inference** costs.
 - While LoRA makes training the model easier, it does not change the fact that we still need to **load a very large model**.
 - Training small models still requires enterprise-grade GPUs.
 - During inference, there is essentially **no gain in performance**.
- The solution to large models is making them smaller → **quantization**.

QLoRA: Quantization

- Quantization refers to dropping the size (not number) of the LLM parameters.
- Normally, each parameter is stored in a 16 bit floating point.
- Quantization can drop them to usually 4 bits, resulting in $\times 4$ less GPU memory and inference costs.



Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

QLoRA: Quantization

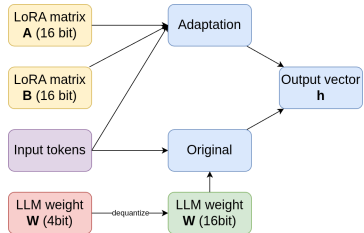
- Quantization is one of the **most useful tools** at your disposal.
- Allows for training and inference in consumer-grade hardware.
- We will return to this in Module 5.



Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

QLoRA: Concept

- Load a 4-bit quantized model using a normal approximation of weights.
- Run LoRA in full precision, **dequantizing** the model weights **one at a time**.
- Continue as normal.



QLoRA: Algorithm

▷ *====Initialization=====*

◁

▷ *Same as before*

◁

▷ *====Backpropagation=====*

◁

for all $X \in \text{input_embeds}$ **do**

$$\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{A} \mathbf{B}$$

$\mathbf{H} = \text{DEQUANT}(\mathbf{W}) X + \Delta \mathbf{W} X$ ▷ *Output = original + adaptation*

▷ *====End of training=====*

◁

▷ *Only attempt dynamic merge. Ignore this advice at your own peril, or if you really know what you are doing.*

◁